

ECOTE Summer 2026

Final Project Report

Scientific Calculator Interpreter

Institution	Warsaw University of Technologies
Semester	Summer 2026
Author and Group	Davit Afyan, Group 103
Subject	Compiling Techniques

I. General Overview and Assumptions

The objective of this project is to design and implement a modular interpreter for a scientific calculator. The system supports complex arithmetic expressions, user-defined variables, and a sufficient library of mathematical functions.

- Language: Python 3.12
- Lazy Evaluation: the interpreter uses a "pull" architecture - the `SourceReader` reads exactly one character at a time, and the `Scanner` produces tokens only when requested by the `Parser`. This satisfies Regulation 13.2.
- Function Registry: all supported functions and their metadata are defined in `functions.py` (a Python dictionary), decoupled from the parser.
- Variable Constraints: variable names cannot shadow reserved function names; a configurable maximum length is enforced at scan time via the `--max-var-len` command-line argument.
- Manual Text Processing: no built-in regular-expression modules or parsing libraries are used. All lexical analysis and syntax parsing are implemented manually, satisfying Regulation 12.
- Error Reporting: every error carries the exact line and column at which it was detected, formatted as "`<Kind> at Line L, Column C: <message>`".
- Output: the result of every statement (assignment or bare expression) is printed to standard output in order of execution.
- Input: the program accepts an optional file path as a positional argument; if omitted it reads from `stdin` (interactive REPL mode).

Supported number format: integers and floating-point literals with an optional leading dot (e.g. `.5 = 0.5`). Output is formatted to at most 11 decimal places with trailing zeros stripped; whole-number results are shown without a decimal point.

II. Functional Requirements

2.1 EBNF Grammar

The grammar is left-factored to resolve the ambiguity between variable identifiers and function calls. Unary negation has lower precedence than exponentiation (so $-2^2 = -(2^2) = -4$).

```

program    = { statement } ;
statement  = assignment | expr_stmt ;
assignment = IDENTIFIER "=" expression NEWLINE/EOF ;
expr_stmt  = expression NEWLINE/EOF ;
expression = term { ( "+" | "-" ) term } ;
term       = factor { ( "*" | "/" ) factor } ;
factor     = unary [ "^" factor ] ;           (* right-associative *)
unary      = "-" unary | power ;             (* lower prec than ^ *)
power      = primary [ "^" factor ] ;        (* used inside unary *)
primary    = NUMBER
            | "(" expression ")"
            | IDENTIFIER [ "(" [ arg_list ] ")" ] ;
arg_list   = expression { "," expression } ;
IDENTIFIER = letter { letter | digit | "_" } ;
NUMBER     = digit { digit } [ "." { digit } ]
            | "." digit { digit } ;

```

2.2 Operator Precedence (lowest to highest)

Operator(s)	Associativity	Example
+ -	Left	$1 + 2 - 3$
* /	Left	$6 / 2 * 3 \rightarrow (6/2) * 3 = 9$
Unary -	Right	$-2^2 \rightarrow -(2^2) = -4$
^	Right	$2^3^2 \rightarrow 2^{(3^2)} = 512$

2.3 Semantic Rules

- If an identifier is immediately followed by '(', it is a function call; otherwise it is a variable reference.
- Function names are reserved and cannot be used as variable names (SemanticError).
- Referencing an undefined variable raises SemanticError.
- Wrong argument count for a function raises SemanticError.
- Variable name length exceeding --max-var-len raises ConstraintError.
- Mathematical domain violations (e.g. $\sqrt{-1}$, $\log(0)$, even root of negative) raise RuntimeError_.
- Division by zero raises RuntimeError_.

2.4 Supported Functions

Function(s)	Args	Description
sin, cos, tan	1	Trigonometric (radians)
asin, acos, atan	1	Inverse trigonometric; asin/acos domain: [-1, 1]
atan2	2	atan2(y, x) - full quadrant arc-tangent
sinh, cosh, tanh	1	Hyperbolic functions
sqrt	1	Square root; negative argument → RuntimeError_
root	2	root(x, n) - n-th root; even root of negative → error
pow	2	pow(base, exp) - exponentiation
exp	1	e ^x
log / ln	1	Natural logarithm; arg must be positive
log10	1	Base-10 logarithm
log2	1	Base-2 logarithm
logb	2	logb(x, base) - arbitrary-base logarithm
abs	1	Absolute value
floor, ceil	1	Floor / ceiling
round	1	Mathematical round-half-up

III. Implementation

3.1 General Architecture

The system follows a strict pipeline (pull) architecture. Data flows character-by-character through four layers, each layer consuming the output of the previous on demand:

SourceReader → **Scanner** → **Parser** → **Evaluator**

- **SourceReader** (source_reader.py): wraps any stream (file or stdin) and exposes next_char(), reading exactly one character per call. Maintains line and col counters for precise error reporting.
- **Scanner** (scanner.py): pulls characters from SourceReader and produces Token objects one at a time via next_token(). Implements state-machine logic to recognize numbers, identifiers, operators, and delimiters. Enforces max_var_len at lexical time.
- **Parser** (parser.py): recursive-descent parser implementing the EBNF grammar. Pulls tokens from the Scanner and builds an immutable Abstract Syntax Tree. Uses a single token of lookahead to distinguish assignments from expression statements.
- **Evaluator** (evaluator.py): tree-walking interpreter that traverses the AST, maintains a symbol table (dict mapping names → floats), dispatches math to the Function Registry, and collects output lines.
- **Exception hierarchy** (exceptions.py): LexicalError, SyntaxError_, SemanticError, ConstraintError, RuntimeError_ - all carry line and col.
- **Function Registry** (functions.py): a Python dict mapping function names to (arg_count, callable). Adding a new function requires only a single registry entry.
- **Entry point** (calculator.py): parses CLI arguments (input_file, --max-var-len), opens the stream, and orchestrates the pipeline.

3.2 Data Structures

- Symbol Table: Python dict[str, float] stored in the Evaluator. Persists across all statements within one execution session.
- Function Registry: Python dict[str, tuple[int, callable]] in functions.py. RESERVED_NAMES = frozenset(registry.keys()) prevents function shadowing.
- Token: Python dataclass with fields type (TokenType enum), value (str | float), line, col.
- AST node hierarchy: all nodes inherit from ASTNode. Leaf nodes: LiteralNode, VariableNode. Unary: NegateNode. Binary: AddNode, SubNode, MulNode, DivNode, PowNode. Call: FuncCallNode. Statements: AssignNode, ExpressionStatementNode.

3.3 Module Descriptions

SourceReader

Wraps any stream object. next_char() calls stream.read(1), increments col on each character, resets col and increments line on newlines, and returns None at EOF. This guarantees strict lazy evaluation - no buffering of lines or file contents.

Scanner

Maintains a one-character lookahead buffer primed at construction. next_token() skips whitespace (space, tab, carriage return), then dispatches on the current character:

1. Newline → NEWLINE token (statement separator).
2. # → comment: consume until end of line, then tail-call next_token().
3. Single-char operators (+-*/^(),=) → corresponding TokenType.
4. Digit or dot → _scan_number(): accumulates digits and at most one decimal point, raising LexicalError on a second dot.
5. Letter or underscore → _scan_identifier(): accumulates alphanumeric/underscore characters, then checks length against max_var_len (ConstraintError if exceeded).
6. Anything else → LexicalError.

Parser

Implements the EBNF grammar as a set of mutually recursive methods. The key design decision is that unary minus is parsed at a lower precedence level than exponentiation:

- _parse_unary() matches optional leading minus signs and delegates to _parse_power().
- _parse_power() calls _parse_primary() and then optionally handles a right-associative '^' by recursing into _parse_factor().
- This two-level split ensures -2^2 parses as $\text{NegateNode}(\text{PowNode}(2,2)) = -4$, not $\text{PowNode}(\text{NegateNode}(2),2) = 4$.

Statement disambiguation uses one token of lookahead: after consuming an IDENTIFIER, if the next token is EQUALS the statement is an assignment; otherwise parsing continues as an expression statement using the already-consumed identifier as a VariableNode.

Evaluator

`_eval_statement()` dispatches on node type. `_eval_assign()` checks for reserved names and the length constraint before evaluating the RHS and updating the symbol table. `_eval_expr()` is a structural recursion over the AST. Division eagerly checks for a zero divisor before performing the operation. Function calls look up the registry, verify arity, evaluate arguments left-to-right, then call the registered callable.

`_format_number(value)` converts a float to a string: if `value == int(value)` it returns a plain integer string; otherwise it formats to 11 decimal places and strips trailing zeros.

3.4 Input / Output Description

Command-Line Interface

```
python3 calculator.py [input_file] [--max-var-len N]
```

- `input_file` (optional): path to a text file; one expression or assignment per line.
- `--max-var-len N` (optional): positive integer restricting variable name length. By default it is set to 50 characters.
- If `input_file` is omitted and `stdin` is a terminal, the program enters interactive REPL mode (variables persist across lines).

Output Format

- Assignment statement `x = <expr>`: prints "`x = <value>`"
- Bare expression `<expr>`: prints "`<value>`"
- Numbers: whole results have no decimal point; fractional results have at most 11 decimal places with trailing zeros stripped.
- Errors: printed to `stderr` in the form "`<Kind> at Line L, Column C: <message>`"; the program exits with code 1.

Execution Examples

```
$ python3 calculator.py input.txt --max-var-len 8
radius = 5
area = 78.5
```

```
$ echo 'sin(3.14159/2)' | python3 calculator.py
0.999999999998
```

3.5 Design Decisions and Notes

- No external parsing or regex libraries are used, satisfying Regulation 12.
- The scanner enforces `max_var_len` eagerly (at lex time), so even a bare long identifier in an expression raises `ConstraintError` before the parser sees it.
- `round()` is implemented as `floor(x + 0.5)` (round-half-up), matching standard mathematical expectation rather than Python's banker's rounding.
- Comments (`#`) are handled entirely inside the scanner and produce no tokens, making the parser independent of comments.

IV. Functional Test Cases

4.1 Test Suite Overview

The test suite is organised into five files and executed with pytest. Coverage spans every layer of the pipeline:

File	Approx. Tests	Focus
test_scanner.py	~35	Token types, positions, comments, errors
test_parser.py	~30	AST shape, precedence, associativity, syntax errors
test_evaluator.py	~70	Arithmetic, variables, functions, error hierarchy
test_comprehensive.py	~200	Edge cases, stress tests, integration scenarios
test_end_to_end.py	~6	CLI, file I/O, lazy evaluation proof, error formatting

4.2 Partial example of Unit Test Cases

Category	Input	Expected Output	Logic Tested	Error Type
Lexical	long_var_name	Token(ID, "long_var_name", L1,C1)	Multi-char identifier scanning	-
Lexical	3.14159	Token(NUM, 3.14159, L1,C1)	Float literal recognition	-
Lexical	.5	Token(NUM, 0.5, L1,C1)	Leading-dot float	-
Lexical	1.2.3	LexicalError	Second dot is illegal	LexicalError
Lexical	a\n b	b at line=2, col=1	Line/col reset after newline	-
Lexical	@ symbol	LexicalError at col	Unknown character	LexicalError
Lexical	abcd (limit=3)	ConstraintError	max_var_len enforcement	ConstraintError
Syntax	(2 + 3) * 4	AST: MulNode(AddNode, 4)	Parens override precedence	-
Syntax	2 + 3 * 4	AST: AddNode(2, MulNode)	Mul > Add precedence	-
Syntax	2^3^2	AST: PowNode(2, PowNode(3, 2))	Right-associative ^	-
Syntax	-2^2	AST: NegateNode(PowNode)	Unary lower prec than ^	-
Syntax	1 + * 2	SyntaxError at col≥4	Consecutive operators	SyntaxError_
Syntax	(1 + 2	SyntaxError	Unclosed parenthesis	SyntaxError_
Semantic	sin = 5	SemanticError	Reserved name as variable	SemanticError
Semantic	custom_log(100)	SemanticError	Undefined function	SemanticError
Semantic	undefined_var	SemanticError	Undefined variable	SemanticError
Semantic	sin(1, 2)	SemanticError	Wrong arity	SemanticError
Constraint	abcd=1 (limit=3)	ConstraintError	--max-var-len enforcement	ConstraintError
Runtime	1 / 0	RuntimeError_ (div/0)	Division by zero	RuntimeError_
Runtime	sqrt(-1)	RuntimeError_	Domain error	RuntimeError_
Runtime	root(-4, 2)	RuntimeError_	Even root of negative	RuntimeError_
Runtime	log(0)	RuntimeError_	Log of non-positive	RuntimeError_
Runtime	asin(2)	RuntimeError_	asin domain [-1,1]	RuntimeError_

4.3 Examples of Integration Test Scenarios

Scenario A - Recursive Dependency and Function Nesting

```
$ python3 calculator.py input.txt --max-var-len 10
```

```
# Input:
initial_v = 10
step = sin(initial_v / 2) + abs(-5)
result = pow(step, root(27, 3))
```

```
# Output:
initial_v = 10
step = 4.04107572533
result = 65.99042501249
```

Scenario B - Right-Associativity and Unary Priority

```
# Input:
2^3^2 - -5*2
```

```
# Output:
522
```

Logic: $3^2=9$, $2^9=512$; then $-(-5*2)=+10$; $512+10=522$.

Scenario C - Error Locality in Multi-Line File

```
# Input (file):
a = 10
b = 20
c = a + (b * sin(0.5)
d = a + b
```

```
# Output:
a = 10
b = 20
Syntax Error at Line 3, Column 23: Expected ')' but got ...
```

Scenario D - Syntax/Coordinate Error

```
# Input:
a = 10 + * 5
```

```
# Output (stderr):
Syntax Error at Line 1, Column 10: Unexpected token '*'
```

Scenario E - Constraint Error

```
# Input (with --max-var-len 3):
theta = 1.5
```

```
# Output (stderr):
Constraint Error at Line 1, Column 1: Variable name 'theta' exceeds maximum length
of 3 characters
```

Scenario F - Missing Function

```
# Input:
result = custom_log(100)

# Output (stderr):
Semantic Error at Line 1, Column 10: Function 'custom_log' is not defined
```

Scenario G - Lazy Evaluation Verification

A mock stream is injected into `SourceReader` and the test asserts that `stream.read()` is called with argument 1 exactly once per character requested. This mathematically proves that no buffering or line-at-a-time reading occurs.

V. Design Corrections Discovered Through Testing

During the development and testing phase, several test cases initially failed, revealing design defects that required targeted corrections. Here are the most notable ones:

5.1 TestRoundingFunctions: `round()` Banker's Rounding vs. Mathematical Round-Half-Up

Failing test

```
def test_round_half_up(self):
    assert run1("round(2.5)") == "3"
```

Root cause

The initial implementation delegated `round()` directly to Python's built-in `round()` function. Python implements banker's rounding (round-half-to-even), so `round(2.5)` returns 2 (rounds to the nearest even integer) rather than 3. The test expected standard mathematical round-half-up semantics.

Correction

A dedicated wrapper `_round_fn` was introduced in `functions.py`:

```
def _round_fn(x):
    import math as _math
    return float(_math.floor(x + 0.5))
```

The registry entry was updated to reference `_round_fn` instead of the built-in. The test now passes: `_round_fn(2.5) = floor(3.0) = 3`.

5.2 test_unary_minus_precedence: -2^2 Returned 4 Instead of -4

Failing test

```
def test_unary_negation_precedence(self):
    # -2^2 must be -(2^2) = -4, not (-2)^2 = 4
    assert run1("-2^2") == "-4"
```

Root cause

The original grammar had unary minus and exponentiation at the same precedence level, both handled inside `_parse_factor()`. This caused `-2^2` to be parsed as `PowNode(NegateNode(LiteralNode(2)), LiteralNode(2)) = (-2)^2 = 4`, which contradicts standard mathematical convention.

Correction

The grammar was restructured to introduce a separate `_parse_unary()` level that sits below exponentiation in the call chain. The key insight is that `_parse_unary()` matches the leading minus and then calls `_parse_power()` (not `_parse_factor()`) for its operand, so that `^` binds to the base number before unary negation is applied:

```
# factor = unary [ '^' factor ]    (right-associative)
# unary  = '-' unary | power
# power  = primary [ '^' factor ]
```

With this split, `-2^2` parses as `NegateNode(PowNode(LiteralNode(2), LiteralNode(2))) = -(4) = -4`. All existing tests for `(-2)^2 = 4` (achieved via explicit parentheses) also continue to pass.

5.3 test_multiple_decimal_points_is_error: 1.2.3 Was Silently Accepted

Failing test

```
def test_multiple_decimal_points_is_error(self):
    # '1.2.3' should raise a lexical error (second dot is illegal)
    with pytest.raises((LexicalError, SyntaxError)):
        all_tokens("1.2.3")
```

Root cause

The initial `_scan_number()` method consumed digits and at most one decimal point, but when it exited the loop after scanning "1.2", the second dot (.) remained in the input stream. Because a bare dot is a valid start of a number (leading-dot float, e.g. .5), the subsequent call to `next_token()` treated .3 as a new, valid NUMBER token. The overall token stream therefore became `[NUMBER(1.2), NUMBER(0.3)]` with no error raised.

Correction

After exiting the main digit-scanning loop, an explicit check was added to detect if the very next character in the lookahead buffer is still a dot:

```
# Inside _scan_number(), after the loop:
if self._peek() == '.':
    raise LexicalError(
        f"Malformed numeric literal: unexpected '.' after '{raw}'",
        self._reader.line, self._reader.col,
    )
```

This guard fires immediately when the scanner has finished consuming "1.2" and peeks at the following character. The test now passes: `all_tokens("1.2.3")` raises `LexicalError` as expected.

End of Report